You said:

exec(3).pyc
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 34s

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

ChatGPT said:

Análise de segurança — `exec(3).pyc`

Malware identificado

Veredito preliminar: MALICIOSO.

A análise estática do bytecode Python 3.13 identificou um programa com comportamento de **downloader/loader de malware (carregador de segundo estágio)**.

O principal comportamento malicioso é buscar instruções em transações da blockchain Solana, obter um endereço remoto, baixar código JavaScript e executá-lo utilizando Node.js.

O arquivo apresenta características de um malware em múltiplos estágios, cujo comportamento final depende de um payload recebido externamente.

1. Identificação do arquivo

Propriedade	Resultado
Arquivo	`exec(3).pyc`
Formato	Bytecode Python compilado
Versão	CPython 3.13
Tamanho	19.185 bytes
SHA-256	`5ab4ca08925d2a4edfca9782ff978f504127bacff4bb08d77875a4a5c9f0ae83`
Execução durante a análise	Não realizada

A análise foi feita pela inspeção do bytecode, das constantes, das funções e das instruções, sem executar o programa.

2. Análise dos comportamentos

2.1. Busca de informações na blockchain Solana

O programa possui a função `_getSignFAddress`, que consulta diversos servidores RPC da rede Solana.

Entre os métodos utilizados está:

```
getSignaturesForAddress
```

Um endereço de carteira está codificado diretamente no programa:

```
BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC
```

O programa procura transações associadas a esse endereço e verifica se elas contêm informações no campo `memo`.

Interpretação: a blockchain está sendo utilizada como mecanismo de distribuição de configurações ou localização da infraestrutura remota do malware. Esse comportamento é altamente suspeito.

2.2. Extração de configurações ocultas

A função `fvzsrnzlh` procura transações que contenham `memo`.

Quando encontra uma informação válida, remove um possível prefixo numérico e tenta interpretá-la como JSON:

```
result = re.sub(r'\[\d+\]\s*', '', memo)
json.loads(result)
```

Isso permite obter configurações armazenadas na blockchain, incluindo um campo chamado `link`.

O programa ainda repete as consultas periodicamente enquanto não encontra os dados esperados.

Interpretação: existe um mecanismo de descoberta dinâmica da infraestrutura de comando e controle (C2), sem a necessidade de manter o endereço final do servidor diretamente no arquivo.

2.3. Verificação da localização da vítima

A função `_isRussianSystem` verifica:

- Nome de usuário do sistema.
- Configurações de idioma e localização.
- Fuso horário configurado.
- Horário UTC local.
- Configuração regional do Windows e do Linux.

Caso identifique determinadas características associadas à Rússia, o programa interrompe sua execução.

Interpretação: trata-se de uma técnica de restrição geográfica, frequentemente utilizada por operadores de malware para evitar infectar determinados territórios.

Há uma particularidade: a verificação do deslocamento UTC entre +2 e +12 horas é bastante abrangente e pode excluir sistemas de outros países também.

2.4. Download e instalação do Node.js

A função `download_nodejs` baixa o Node.js versão 22.9.0, escolhendo a distribuição conforme o sistema operacional e a arquitetura.

São contemplados:

- Windows (x86 e x64).
- macOS (x64 e ARM64).
- Linux (x64 e ARM64).

Os arquivos são extraídos no diretório pessoal do usuário.

Essa etapa é particularmente suspeita porque o Node.js será utilizado para executar um segundo estágio de código recebido remotamente.

O download do Node.js, isoladamente, não é malicioso. O problema é sua utilização no restante do fluxo.

2.5. Comunicação com o servidor remoto

A função `scyzzvvy` recebe um endereço URL previamente obtido da blockchain e realiza uma requisição HTTP GET.

Um detalhe importante é o envio de informações sobre o sistema operacional:

```
headers = {
    "os": platform.system().lower()
}
```

A resposta HTTP é interpretada de três maneiras:

- Corpo da resposta: código JavaScript codificado em Base64.
- Cabeçalho `ivbase64` : informação relacionada a um vetor de inicialização.
- Cabeçalho `secretkey` : informação relacionada a uma chave secreta.

Os nomes dos dois cabeçalhos são inicialmente armazenados codificados em Base64:

Valor encontrado	Texto decodificado
`aXZiYXNlNjQ=`	`ivbase64`
`c2VjcmV0a2V5`	`secretkey`

Interpretação: o servidor fornece tanto o código quanto parâmetros que podem ser utilizados pelo segundo estágio para operações criptográficas.

2.6. Execução do código JavaScript recebido

Esta é uma das evidências mais fortes de comportamento malicioso.

A função `callback` constrói um arquivo chamado:

```
i.js
```

O conteúdo produzido segue esta estrutura:

```
var https = require("https");

const secretKey = '...';

const _iv = Buffer.from('...', "base64");

eval(atob('...'));
```

A instrução mais importante é:

```
eval(atob('...'));
```

Ela realiza duas operações:

1. Decodifica o conteúdo recebido em Base64.
2. Executa o resultado como código JavaScript.

Portanto, o programa permite que um servidor remoto envie instruções arbitrárias para execução na máquina da vítima.

As variáveis `secretKey` e `_iv` também ficam disponíveis para esse código.

O arquivo JavaScript é executado com:

```
subprocess.Popen(
    [node_exe, js_file_path],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL
)
```

Dois aspectos merecem destaque:

- A saída padrão e os erros são descartados, dificultando a observação da execução.
- No Windows, o programa utiliza `CREATE_NO_WINDOW`, evitando a abertura de uma janela de console.

Interpretação: esse comportamento caracteriza execução de payload remoto, uma técnica típica de loaders e trojans.

2.7. Controle de execução e tentativa de evitar detecção

O programa também apresenta algumas técnicas auxiliares:

Comportamento	Finalidade provável
Espera inicial de 10 segundos	Atrasar a execução
Verificação regional	Evitar determinados alvos
Consultas repetidas à Solana	Aguardar instruções remotas
Arquivo `init.json`	Controlar a frequência de execução
Nomes de funções pouco descritivos	Dificultar a análise
Execução sem console	Reduzir a visibilidade do processo

O arquivo `init.json` armazena uma data e estabelece um intervalo de aproximadamente dois dias antes de permitir outra execução. Não identifiquei, entretanto, um mecanismo convencional de inicialização automática junto ao sistema operacional.

3. Indicadores de comprometimento (IoCs)

Os seguintes elementos podem ser utilizados em uma investigação de segurança:

Identificador da amostra

```
SHA-256:
5ab4ca08925d2a4edfca9782ff978f504127bacff4bb08d77875a4a5c9f0ae83
```

Endereço Solana utilizado na busca de configurações

```
BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC
```

Infraestrutura RPC consultada

```
https://api.mainnet-beta.solana.com
https://solana-mainnet.gateway.tatum.io
https://go.getblock.us/
https://solana-rpc.publicnode.com
https://api.blockeden.xyz/
https://solana.drpc.org
https://solana.leorpc.com/
https://solana.api.onfinality.io/public
https://solana.api.pocket.network/
```

Artefatos criados ou utilizados

```
~/.init.json
i.js
node-v22.9.0-*
```

O endereço final do servidor que entrega o payload não pode ser determinado somente pela análise estática deste arquivo, pois ele é obtido dinamicamente a partir da blockchain.

4. Avaliação final

CLASSIFICAÇÃO FINAL MALWARE

Malicioso

Categoria principal: Trojan Downloader / Loader.

Subcategoria: Loader de múltiplos estágios com infraestrutura de comando e controle baseada em blockchain.

Justificativas:

1. Pesquisa periodicamente informações em transações de uma carteira Solana específica.
2. Utiliza dados obtidos na blockchain para descobrir um servidor remoto.
3. Baixa conteúdo executável desse servidor.
4. Prepara um ambiente Node.js para executar o conteúdo recebido.
5. Utiliza `eval(atob(...))` para executar código JavaScript remoto.

6. Descarta a saída do processo e oculta a janela de execução no Windows.
7. Implementa verificações regionais para restringir a execução.

5. Conclusão

O arquivo `exec(3).pyc` é um malware, não um programa benigno.

Sua função principal é atuar como um **carregador de malware (Malware Loader/Downloader)**, obtendo instruções por meio da blockchain Solana e executando um segundo estágio JavaScript recebido de uma infraestrutura remota.

Há indícios de uma infraestrutura voltada ao ecossistema de criptomoedas, especialmente pela utilização da Solana e pelas variáveis `secretKey` e `_iv`. Entretanto, não é possível afirmar que o payload final seja um ladrão de carteiras (*crypto stealer*) ou um drainer, pois esse segundo estágio não está incorporado na amostra analisada.

Confiança na classificação como malware: alta.

Recomendação: não execute o arquivo. Caso já tenha sido executado, investigue a presença de `i.js`, dos diretórios Node.js criados no diretório pessoal e das conexões de rede realizadas pelo processo.

‹/›

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal é **Trojan Downloader/Loader (Carregador de Malware)**.

Sua função é localizar um servidor remoto por meio da blockchain Solana, baixar um payload malicioso e executá-lo usando Node.js.